

Lab Report – 5

CS – 344 | Operating System Lab | 2025

Group No. 24

TASK – 5.1: IMPLEMENT DOUBLY-INDIRECT BLOCK SUPPORT

Objective

The primary objective of this project is to overcome the inherent file size limitation in the standard xv6 file system. This is achieved by modifying the inode structure and its related functions to include support for a doubly-indirect block pointer.

Enhancing xv6 capabilities

The standard xv6 file system design limits file sizes due to its inode structure, which contains 12 direct block pointers and one singly-indirect block pointer. With a block size of 1024 bytes, this architecture caps the maximum file size at 268 blocks (12 direct + 256 indirect), or approximately 268 KB.

The new inode structure now consists of **11 direct pointers**, one **singly-indirect pointer**, and one **doubly-indirect pointer**. While the direct and singly-indirect pointers contribute 11 and 256 blocks respectively. This single doubly-indirect pointer can address $256 \times 256 = 65,536$ blocks. Summing these up, the total number of addressable blocks becomes $11 + 256 + 65,536 = 65,803$ blocks. This modification elevates the maximum file size from approximately 268 KB to **65,803 KB** approximately 64 MB.

Design and Implementation

High Level Description

To accommodate the new doubly-indirect pointer without changing the overall size of the on-disk inode, the number of direct block pointers was reduced from 12 to 11.

The `bmap()` function, which translates a file's logical block number into a physical disk block address, was extended. It now includes a third tier of logic to handle block numbers that fall within the doubly-indirect range with a two-step lookup.

The `itrunc()` function, responsible for freeing a file's blocks, was updated to recursively free the top-level doubly-indirect block to prevent disk space leaks.

Lab Report – 5

CS – 344 | Operating System Lab | 2025

Group No. 24

Implementation

1. Definition of new NDOUBLY_INDIRECT (kernel/fs.h):

```
#define NDIRECT 11 // changes from 12 to 11 for assignment
#define NINDIRECT (BSIZE / sizeof(uint))
#define NDOUBLY_INDIRECT (NINDIRECT * NINDIRECT)
#define MAXFILE (NDIRECT + NINDIRECT + NDOUBLY_INDIRECT)
```

The `addrs` array in `struct dinode` was resized to `uint addrs[NDIRECT+2];`

2. Modifying `struct inode` (kernel/file.h):

The `addrs` array in `struct inode` was resized to `uint addrs[NDIRECT+2];`

3. Updated FSIZE in (kernel/param.h): `#define FSSIZE 200000`

4. Modifying kernel/fs.c:

In `bmap()`, a new if block was added to handle logical block numbers greater than `NDIRECT + NINDIRECT`. This new code calculates indices to traverse the doubly-indirect structure.

In `itrunc()`, a new section was added with nested loops to iterate through the doubly-indirect block, free all associated data and singly-indirect blocks, and finally free the doubly-indirect block itself.

```
bn -= NINDIRECT;
if(bn < NDOUBLY_INDIRECT){
    uint outer_index = bn / NINDIRECT;
    uint inner_index = bn % NINDIRECT;

    // Load doubly-indirect block, allocating if necessary
    if((addr = ip->addrs[NDIRECT+1]) == 0)
        ip->addrs[NDIRECT+1] = addr = balloc(ip->dev);

    bp = bread(ip->dev, addr);
    a = (uint*)bp->data;

    // Get address of the inner indirect block
    if((addr = a[outer_index]) == 0){
        a[outer_index] = addr = balloc(ip->dev);
        log_write(bp);
    }
    brelse(bp);

    // Load the inner indirect block
    bp = bread(ip->dev, addr);
    a = (uint*)bp->data;

    // Get the actual data block address
    if((addr = a[inner_index]) == 0){
        a[inner_index] = addr = balloc(ip->dev);
        log_write(bp);
    }
    brelse(bp);
    return addr;
}
```

```
if(ip->addrs[NDIRECT+1]){
    bp = bread(ip->dev, ip->addrs[NDIRECT]);
    a = (uint*)bp->data;

    for(j = 0; j < NINDIRECT; j++){
        if(a[j]){
            bp2 = bread(ip->dev, a[j]);
            a2 = (uint*)bp2->data;

            for(k = 0; k < NINDIRECT; k++){
                if(a2[k])
                    bfree(ip->dev, a2[k]);
            }
            brelse(bp2);

            bfree(ip->dev, a[j]);
        }
    }
    brelse(bp);

    bfree(ip->dev, ip->addrs[NDIRECT+1]);
    ip->addrs[NDIRECT+1] = 0;
}
```

[complete code file is attached]

Lab Report – 5

CS – 344 | Operating System Lab | 2025

Group No. 24

Test File: [code file is attached]

To validate the implementation, the **bigfile.c** user program was created. Its purpose is to test the file system's ability to handle files that exceed the original size limit.

1. **Write Phase:** The program creates a file named **bigfile** and writes a large number of blocks (e.g., 3000), which is well beyond the old 268-block limit. Each block is filled with a deterministic, predictable data pattern based on its block number.
2. **Verification Phase:** After writing is complete, the program closes and re-opens the file for reading. It reads the file block-by-block, regenerating the expected data pattern for each block and comparing it with the data read from the file.
3. **Outcome:** If the contents of all blocks match perfectly, the program prints a success message. If any mismatch is found, it reports an error, indicating a failure in the file system logic.

Output:

```
$ bigfile
bigfile: starting file I/O test
bigfile: writing 3000 blocks to bigfile...
bigfile: written 500/3000 blocks
bigfile: written 1000/3000 blocks
bigfile: written 1500/3000 blocks
bigfile: written 2000/3000 blocks
bigfile: written 2500/3000 blocks
bigfile: successfully wrote 3000 blocks
bigfile: verifying 3000 blocks in bigfile..
bigfile: verified 500/3000 blocks
bigfile: verified 1000/3000 blocks
bigfile: verified 1500/3000 blocks
bigfile: verified 2000/3000 blocks
bigfile: verified 2500/3000 blocks
bigfile: successfully verified 3000 blocks
bigfile: test completed successfully
```

The output confirms that the test program successfully created, wrote to, and verified a file of 3000 blocks. This size is significantly larger than the original xv6 maximum.

TASK – 5.2: IMPLEMENT SYMBOLIC LINKS (SOFT LINKS)

Objective

The objective of this project is to extend the xv6 file system to support symbolic (soft) links.

A **symbolic link** (or symlink) is a special type of file whose only job is to point to another file or directory by its name.

Lab Report – 5

CS – 344 | Operating System Lab | 2025

Group No. 24

Enhancing xv6 capabilities

The standard xv6 file system supports regular files, directories, and hard links. However, hard links are limited: they cannot point to directories and cannot span across different file systems. This project enhances xv6 by introducing symbolic links, which overcome these limitations.

Design and Implementation

High Level Description

The solution integrates symbolic links by introducing a new file type, T_SYMLINK. A new system call, `sys_symlink`, is created to allocate an `inode` of this type and write the target path into the inode's data blocks. The core logic resides in the modified `sys_open` function, which now checks an inode's type. If it encounters a T_SYMLINK, it reads the target path and recursively re-runs the path resolution logic on the new path. To prevent kernel stack overflow from circular links, this recursion is limited to a depth of 10.

Implementation

1. Adding New Constants:

- A new file type constant was added to kernel/stat.h to identify symbolic links.
- A new flag was added to kernel/fcntl.h to give user programs control over link following.

```
#define T_SYMLINK 4
```

```
#define O_NOFOLLOW 0x800
```

2. Changes in kernel/sysfile.c:

- Following was added in `create()` function to handle the case when we try to create already created symFile:

```
if((ip = dirlookup(dp, name, 0)) != 0){
    iunlockput(dp);
    ilock(ip);
    /*----- Changes made for Task-5.2 -----*/
    if(type == T_SYMLINK) {
        return ip;
    }
    /*-----*/
    if(type == T_FILE && (ip->type == T_FILE || ip->type == T_DEVICE))
        return ip;
    iunlockput(ip);
    return 0;
}
```

Lab Report – 5

CS – 344 | Operating System Lab | 2025

Group No. 24

- The implementation, `sys_symlink`, was added. It uses the existing `create()` helper function to make a new inode and then writes the target path into it.

```
uint64
sys_symlink(void)
{
    char target[MAXPATH], path[MAXPATH];
    struct inode *ip;
    int len;

    argstr(0, target, MAXPATH);
    argstr(1, path, MAXPATH);

    begin_op();

    ip = create(path, T_SYMLINK, 0, 0);
    if(ip == 0){
        end_op();
        return -1;
    }

    len = strlen(target);
    if(writei(ip, 0, (uint64)target, 0, len + 1) != len + 1){
        ip->nlink = 0;
        iupdate(ip);
        iunlockput(ip);
        end_op();
        return -1;
    }

    iunlockput(ip);
    end_op();
    return 0;
}
```

- The `sys_open` function was updated to handle the `T_SYMLINK` type. If a path resolves to a symbolic link and the `O_NOFOLLOW` flag is not present, the kernel reads the target path from the link's data and restarts the name resolution process. A loop counter prevents infinite recursion.

Lab Report – 5

CS – 344 | Operating System Lab | 2025

Group No. 24

```
if((ip->type == T_SYMLINK) && !(omode & O_NOFOLLOW)){
    int loop_count = 0;
    while(ip->type == T_SYMLINK && loop_count < 10){
        if(readi(ip, 0, (uint64)path, 0, ip->size) != ip->size){
            iunlockput(ip);
            end_op();
            return -1;
        }
        // Ensure the path is null-terminated.
        path[ip->size] = 0;
        iunlockput(ip);

        if((ip = namei(path)) == 0){
            end_op();
            return -1;
        }

        ilock(ip);
        loop_count++;
    }
    if(loop_count >= 10){
        printf("<error>: cycle detect in symlink!\n");
        iunlockput(ip);
        end_op();
        return -1;
    }
}
```

[complete code file is attached]

- Other necessary changes were made in kernel/syscall.h, kernel/syscall.c, user/user.h and user/sys.pl to implement symlink system call properly.

Test File: [code file is attached]

To validate the implementation, symlinktest.c performs a series of checks.

First, it creates a file t.txt with known content. It then creates a symbolic link link1 pointing to t.txt and reads from link1 to verify that the system correctly follows the link and returns the content of t.txt.

Next, it tests the O_NOFOLLOW flag by opening link1 with it, confirming that this reads the link's content (the path "t.txt") instead of following it.

Finally, and most critically, it creates a circular link (a -> b and b -> a) and attempts to open it. This is designed to test the loop detection mechanism, ensuring the kernel call fails gracefully with an error instead of crashing.

Lab Report – 5

CS – 344 | Operating System Lab | 2025

Group No. 24

Output:

```
$ symlinktest
Starting symlinktest...

--- Test 1: Failed File and Symlink Creation ---
Attempting to create and write to a regular file 't.txt'...
$ CCESS: 't.txt' created and written to successfully.

Attempting to create a symbolic link 'link1' that points to 't.txt'...
SUCCESS: Symbolic link 'link1' created.

--- Test 2: Following a Symbolic Link ---
Opening 'link1' without O_NOFOLLOW. This should open the target file 't.txt'.
SUCCESS: Read from followed link 'link1'. Content: 'hello'
'-----

--exec t 3: Opening a Symlink with O_NOFOLLOW ---
Opening 'link1' with O_NOFOLLOW. This should open the link itself, not the target.
Reading the content of the symlink file...
SUCCESS: The content of the symlink is its target path: 't.txt'
-----

--- Test 4: Symbolic Link Loop Detection ---
Creating a symlink loop: 'a' -> 'b'...
Creating a symlink loop: 'b' -> 'a'...
Link loop 'a' <-> 'b' created successfully.

Attempting to open 'a'. The kernel should detect the loop and prevent opening.
<error>: cycle detect in symlink!
SUCCESS: As expected, open('a') failed, preventing an infinite loop.
-----

symlinktest finished successfully!
```

References

<https://pdos.csail.mit.edu/6.828/2024/xv6/book-riscv-rev4.pdf>

https://xiayingp.gitbook.io/build_a_os

<https://www.geeksforgeeks.org/linux-unix/how-to-symlink-a-file-in-linux/>

Files

https://drive.google.com/drive/folders/1Xq0663fKCqFWhDd8Edj2u_mZuz4wlF7g?usp=sharing